

Phase 3a Study Guide — the ML layer of the UAV twin

A plain-language walk-through of what the four notebooks do, why, and how they fit

Asick Jahir · with Claude (Anthropic)

September 2026 · Phase 3c complete, appendix A added

Status update — 15 September 2026. All six notebooks are complete, both models are pickled and loaded in the worker, and Phase 3c is verified end-to-end. This document remains the pedagogical walkthrough for what each notebook was designed to do and why — the achieved metrics and the operational reference live in Appendix A at the end and in the companion doc `phase-3c-complete.md`.

Part 1 — Where Phase 3 sits and what it's for

The twin you have running today (Phase 1c) has three moving parts:

- **A vehicle** (ArduPilot SITL for now, a real UAV later) that emits telemetry.
- **A physics model** (the Modelica FMU) that predicts what a *healthy* vehicle's motors and battery should look like given the same inputs.
- **A dashboard** that shows both, side by side.

Phase 3 adds a fourth part: **an ML layer** that reads the *residuals* — the difference between what the sensor says and what the physics predicts — and turns them into three decisions.

- *Is a motor showing signs of failure right now?* → Model A (bearing/motor wear classifier)
- *How much useful life is left in the battery pack?* → Model B (SoH / RUL Gaussian process)
- *Is anything anomalous happening that neither A nor B recognises?* → Model C (safety-net anomaly detector — deferred until real fleet data exists)

The four notebooks 01-04 you now have are the **data-preparation and first-model-training** work for this ML layer. They don't touch the running twin yet — that happens in Phase 3c (notebook 06). Here they build the substrate that makes the twin production-worthy.

1.1 Why do this at all?

The Phase 1c twin is already useful in two ways: it monitors the physical state faithfully, and its FMU-based predictions can flag gross faults through simple residual thresholds. But three things it *cannot* do yet:

1. **Distinguish “the battery is old” from “this specific flight was hard on the battery”.** SoH is a slow-varying state; the FMU integrates it, but the twin has no way to say “given this SoH, expect N flights before EoL.” A Gaussian process trained on real aging data does that with an uncertainty band.
2. **Detect faults faster than a rule-based threshold.** A motor bearing developing a defect changes the *shape* of the current signature before the *amplitude* crosses a threshold. XGBoost on hand-crafted features can learn that shape.
3. **Catch novel failures it hasn’t seen before.** Neither the FMU nor a supervised classifier can flag a failure mode that wasn’t in the training set. An anomaly detector (Model C) is the catch-all for the unknown unknowns.

1.2 What’s the goal of Phase 3a specifically?

Phase 3a is the *data exploration + first-model training* sub-phase. Four things need to be true before we move into Phase 3b (Model B) and Phase 3c (worker integration):

- We have concrete answers about whether the open datasets we’re using actually carry the signal we want to learn.
- We have a first working ML model (Model A) with an honest cross-validation score.
- We know which features carry the signal and which are noise.
- The whole ML pipeline runs on your laptop end-to-end, with a synthetic fallback so you can develop before the real 5+ GB datasets are downloaded.

The four notebooks deliver all four.

Part 2 — What each notebook contributes

2.1 Notebook 01 — NASA IMS bearing exploration

Question it answers. *When a bearing is developing a defect, what does the sensor signal look like over time?*

Data. The IMS bearing dataset from the University of Cincinnati’s Center for Intelligent Maintenance Systems, mirrored on NASA’s Prognostics Center of Excellence data repository. Four bearings on a shaft, run continuously for about a week each, until one develops an outer-race fault and fails. Sampled at 20.48 kHz across accelerometer channels.

What you’re looking at. In this dataset the sensor is a physical accelerometer glued to the bearing housing. In our twin, the equivalent sensor is the ESC-reported motor current — because a bearing defect on a motor rotor shows up as a modulation of the motor’s air-gap flux, and thus as sidebands in the stator current. The physical mechanism is the same; the wire we listen to is different. This is called *motor-current signature analysis* (MCSA) and it’s a mature technique in industrial motor prognostics.

What the notebook actually does. It loads 60 one-second vibration snapshots spread evenly across a bearing’s life from new to failed, extracts eight standard prognostic features per snapshot (RMS, kurtosis, crest factor, spectral energy in the 2-5 kHz bearing-resonance band, etc.), and shows how each feature evolves as the bearing wears out. The eight features come straight out of the industrial-vibration literature; nothing exotic.

What “good” looks like.

- Raw signal plots: healthy trace is a smooth low-amplitude noise band; failing trace is visibly spiky. The spikes are the balls rolling over the developing defect.
- Feature-progression plots: RMS is flat until ~60% of life then climbs; kurtosis (a measure of “spikiness”) transitions from Gaussian noise (~ 0) to strongly impulsive (> 2) around the same point; spectral energy in the 2-5 kHz band grows by two orders of magnitude by end-of-life.
- Spectrum plot: the failing spectrum has a clear energy hump in the 2-5 kHz range that isn’t there in the healthy one. That hump *is* the fault.

Why this matters for the twin. IMS gives us the ground truth for the *shape* of a bearing wear signal. Model A ultimately doesn’t train on IMS directly (we don’t have accelerometers on our UAV motors); it trains on motor-current features. But we validated with IMS that the underlying feature space — RMS + kurtosis + crest factor + spectral energy — carries the wear signal cleanly. That means the same features computed on motor current *should* work.

2.2 Notebook 02 — ALFA UAV fault flights

Question it answers. *On a real UAV, at 10 Hz motor telemetry cadence, does a fault produce a signature clear enough to train a classifier on?*

Data. The ALFA dataset from Carnegie Mellon’s AirLab — 47 real fixed-wing UAV flights with deliberately-injected control-surface and motor faults, labelled at millisecond accuracy. Distributed as ROS bag files, ~2 GB total. Permissively licensed (CC-BY). The synthetic fallback in the notebook produces multirotor telemetry with the same schema our MAVLink bridge produces on the twin’s MQTT bus.

Why the switch from vibration to current, from long time-scale to short time-scale. IMS runs bearings for days. ALFA fault-inject on a scale of seconds within a five-minute flight. And the sensor becomes the ESC current, not an accelerometer. This is a deliberate move down the “how close to real deployment” gradient: Notebook 01 established shape from a lab bench; Notebook 02 establishes shape from a real airborne UAV.

What the notebook actually does. It loads 20 flights (a mix of healthy and faulty), plots two representative traces side by side (four coloured lines for four motors — healthy has them overlapping, faulty has one visibly separating from the others), and then slides a 2-second window across each flight at 1-second stride to extract 28 features per window. Every window becomes a training example for Model A. Twenty flights → about 6000 windows.

What “good” looks like.

- The visual comparison in §2 is the money shot: healthy flight has four motor-current traces that essentially overlap; faulty flight has one motor’s trace clearly diverging from the others at the labelled fault-onset time.
- The feature-progression plots in §4 show `cross_max_asym` (imbalance between highest-current and lowest-current motor) jumping at fault onset and staying elevated for the rest of the fault window.

- The histogram in §6 overlays healthy-window and fault-window distributions of `cross_max_asym`. If the two distributions barely overlap, XGBoost will train easily. On the synthetic corpus we see a 40× separation between healthy and fault means.

Feature engineering strategy. For each 2-second window we compute six features per motor (mean, RMS, standard deviation, peak of current; mean temperature; temperature slope) and four cross-motor features (max asymmetry, current variance, min RPM, max temperature spread). Twenty-four per-motor + four cross-motor = 28. The cross-motor features are the ones that carry most of the signal — a physically-motivated design choice: a healthy multirotor should have balanced motors, so any imbalance is a fault signature by construction.

2.3 Notebook 03 — NASA Li-ion battery aging

Question it answers. *How does Li-ion capacity fade over hundreds of cycles, and what cycle-level features predict remaining useful life?*

Data. NASA PCoE’s canonical Li-ion battery aging set — four 18650 cells (labelled B0005/B0006/B0007/B0018), each cycled continuously in a lab at 24 °C ambient until capacity fades below ~70% of nominal. This is the reference dataset for Li-ion prognostics in academia and industry.

Why this notebook is different from 01 and 02. Battery aging is a *slow* signal — hundreds of cycles, days of testing. Motor-fault detection is a *fast* signal — sub-second responses to millisecond events. So the feature engineering is completely different: we work at the cycle level, not the millisecond level.

What the notebook actually does.

- Loads cycle-by-cycle capacity data for four cells (synthetic fallback produces plausible two-phase aging curves matching the real dataset’s shape).
- Shows the capacity-fade signature: gentle linear region for the first 100-120 cycles, then an accelerating “knee” that ramps to EoL.
- Plots two leading indicators — internal resistance rise and peak-cycle temperature drift — that change measurably *before* SoH crosses common alarm thresholds.
- Computes rolling 5-cycle slope features (rate of capacity decline, rate of resistance rise, rate of temperature drift) that give the model *trajectory* awareness, not just current state.
- Converts cycles to flights using the empirical ratio “1 flight $\approx$ 0.2 equivalent full cycles” (a partial discharge in each flight), producing an RUL curve in the units an operator actually thinks in.

What “good” looks like.

- Four capacity-fade curves that all show the same shape but reach EoL at different cycle counts. B0006 in the notebook ages fastest (~35% higher aging rate); B0007 slowest. This inter-cell variability is *the point* — it’s why we use a Gaussian process for Model B in Phase 3b. A model that ignores this variability will systematically over- or under-estimate RUL.
- Internal-resistance curves that visibly rise 40-60 cycles before SoH crosses 80%. This lead time is what we sell to a customer.
- The flight-based RUL plot decays from ~1000 flights at start-of-life to 0 at EoL, monotonically for a well-behaved cell.

Why this matters for the twin. Notebook 03 establishes the shape of Model B’s *target* variable. In Phase 3b we’ll train a Gaussian process to predict this curve, given the covariates the FMU already outputs (Coulomb-counted SoH, internal resistance, peak temperature). GP’s output isn’t a single number — it’s a distribution, so the twin can honestly report “flights remaining: 47 ± 12 (90% credible interval)”.

2.4 Notebook 04 — Model A XGBoost training

Question it answers. *Does a real classifier trained on the ALFA feature space actually work?*

Data. The Parquet file that Notebook 02 saved (data/alfa_features/flight_windows_*.parquet), plus optionally the IMS feature file from Notebook 01 for a wear-vs-fault comparison. This is the first notebook that consumes another notebook’s output — Notebook 04 does *not* re-run Notebooks 01/02’s work.

What the notebook actually does.

1. Loads the ALFA feature corpus (about 6000 windows $\times$ 28 features + 1 label).
2. Splits into 5 cross-validation folds *by flight* — every fold holds out entire flights, so the model never sees any part of a test flight during training. This is the honest way to evaluate temporal ML; the wrong way (random-split) gives you inflated metrics because the model memorises within-flight noise.
3. Trains an XGBoost gradient-boosted-tree classifier with balanced class weights.
4. Evaluates: cross-validated ROC curve, precision-recall curve, feature-importance ranking.
5. Picks an *operational threshold* — the P(fault) cutoff at which the twin fires an alert. We optimise for precision $\geq 90\%$ first (an operator won’t tolerate false alarms) and take the highest recall available at that precision.
6. Confusion matrix and per-flight visualisation: plot P(fault) over time on one held-out faulty flight, overlaid with the ground-truth fault window. This is the visualisation that eventually goes on the twin’s dashboard.
7. Persists two files to worker/models/:
 - bearing_wear.pkl — the trained model, pickled
 - bearing_wear.json — a companion config the twin worker will use to know the feature order, threshold, and provenance

What “good” looks like.

- Mean cross-validated AUC ≥ 0.95 — on the synthetic corpus we hit 0.999 because the fault signal is nearly separable. Real data will be lower; > 0.85 on real ArduPilot logs is a reasonable target.
- The feature-importance bar chart is dominated by red bars (cross-motor features). The top-3 should all be `cross_max_asym`, `cross_i_var`, `cross_max_temp_diff`. This means the model has learned the physical concept we designed the feature space around.
- The per-flight probability plot shows the classifier’s P(fault) crossing the operational threshold at or slightly *before* the labelled fault onset. A negative “detection lag” is early detection — what we want.

The one honest caveat baked into this notebook. The synthetic data is deliberately easier than reality — no missing telemetry, no radio dropouts, no thermal-runaway edge

cases, no operator-induced anomalies that aren't faults. Re-train on real fleet data before shipping anything to a paying customer. The pipeline stays identical; only the data underneath changes.

Part 3 — Common patterns across the four notebooks

These are the design decisions that recur across every notebook. They aren't accidental — each one exists to solve a specific problem you'll encounter as this project scales.

3.1 Every notebook has a synthetic fallback

Real datasets are big (5 GB IMS, 2 GB ALFA, 500 MB NASA batteries) and take time to download and clean. Every notebook is designed so you can run it end-to-end today, on your laptop, with no downloads. The synthetic fallback produces data with the same *shape* as reality — same feature distributions, same characteristic frequencies, same aging curve topology. The fallback code lives in `utils/*_loader.py`.

The switchover to real data is one line: set an environment variable pointing at the real-data folder, re-run the notebook, nothing else changes. Every plot, every feature, every parquet output regenerates automatically from the real data.

Why this pattern matters. It de-risks the schedule. You can iterate on model design, feature engineering, and evaluation methodology today, and drop in real data later without rewriting anything. It also gives you the ability to reproduce a bug or a decision even when the original data is gone.

3.2 Every notebook produces a Parquet file for the next step

Notebook 01 produces `data/ims_features/*.parquet`. Notebook 02 produces `data/alfa_features/*.parquet`. Notebook 03 produces `data/battery_features/*.parquet`. Notebook 04 consumes Notebook 02's output and produces two model artefacts.

This means every notebook is a *stage* in a pipeline, not a monolith. If you want to change the ALFA feature set, you re-run Notebooks 02 and 04 — Notebook 01 and 03 are untouched. This is the same pattern you already use in your powertech portfolio project.

Parquet is columnar-typed, small, fast to load, and gives us Arrow-compatible schemas we'll need when the twin talks to an eventual cloud analytics backend. CSV works as a fallback if `pyarrow` isn't installed.

3.3 Every model saves both a `.pkl` and a `.json`

The `.pkl` is the trained model — a serialised object the worker can `pickle.load()`. The `.json` is a companion that records the *contract*: what feature names the model expects, in what order, at what threshold to alert, what data it was trained on, and what its cross-validated performance was.

The worker uses the `.json` first — it verifies the feature vector it built matches the model's expected schema before calling `predict_proba()`. This prevents an entire class of pro-

duction failures where you retrain the model, ship a new .pkl file, but forget that the input features shifted.

3.4 Every notebook ends with a “What we learned” and a “Next steps” section

This is deliberate. Six months from now when you come back to this project, or a design-partner engineer wants to understand what you did, the notebook headers alone aren’t enough. The plain-language summary at the end of each notebook is the durable record of the intellectual work.

Part 4 — The bigger picture: how these fit into the twin

4.1 Where the ML layer lives

Recall the six-layer twin architecture from the Phase 1c technical overview. Layers (1) through (6) are:

1. The asset (SITL or a real UAV)
2. The physics model (FMU)
3. Telemetry ingest (MQTT via Mosquitto)
4. Twin state (in-worker for now, Ditto later)
5. Analytics / ML (the worker’s `twin_worker.py`)
6. UX / App (the browser dashboard)

The ML layer is a growth of layer (5). Currently the worker runs the FMU and emits predictions. Phase 3c grew a sibling `ml_runner.py` that loads Models A, B (and later C) and emits its own predictions on the same MQTT topics. From the dashboard’s point of view nothing changes — same JSON schema, same tiles.

4.2 The data flow at runtime

Every 100 ms:

1. Telemetry arrives on `uav/quad-01/telemetry/state`.
2. Worker feeds motor currents into the FMU. FMU returns predicted motor temperature, predicted SoC, etc.
3. Worker computes residuals (measured – predicted) and appends the current sample to a 2-second rolling buffer.
4. When the buffer is full, the worker extracts the 28-feature vector (same features Notebook 02 computes offline) and calls `xgboost_model.predict_proba()`. Output is $P(\text{fault}) \in [0, 1]$.
5. Worker also calls the Gaussian process for battery RUL: input is the FMU’s SoH + covariates, output is a mean and standard deviation for flights-remaining.
6. Worker publishes both to `uav/quad-01/twin/state`, `uav/quad-01/twin/anomaly`, and `uav/quad-01/twin/rul`.
7. Dashboard updates the RUL tile and — if $P(\text{fault})$ crosses the operational threshold — surfaces an alert.

Total added latency: ~2 ms for XGBoost inference, ~5 ms for GP prediction. Well within the 50 ms per-tick budget in the Phase 3 kickoff.

4.3 What happens when a model is unavailable

The worker uses the same fallback pattern as Phase 1c's FMU: if `bearing_wear.pkl` fails to load — file missing, wrong Python version, corrupt — the worker logs it and continues with the FMU's analytical predictor. The twin never goes dark. The dashboard shows "physics mode: analytical fallback" instead of "physics mode: fmu + ml".

Part 5 — How to read the outputs when you run them

5.1 The four “does this look right?” gut checks

For each notebook, the fastest way to know it worked without reading numbers:

Notebook 01 — Look at the raw-vibration plot in §2. Three panels stacked. The top one (healthy) should be a thin flat line. The bottom one (failing) should have visible spikes. If the two look identical, the loader is stuck on a single time-step or the synthetic generator is broken.

Notebook 02 — Look at the two-panel plot in §2. Top panel (healthy flight) should show four coloured lines essentially overlapping — you might not even distinguish them. Bottom panel (faulty flight) should show one line clearly separating from the other three at the dashed vertical (fault-onset) line. If the traces look identical, the fault injection didn't apply.

Notebook 03 — Look at the capacity-fade plot in §2. Four curves that all start near 1.9 Ah, all end below 1.5 Ah, with visible knees. If any curve is flat or noisy, the synthetic aging model or the real-data preprocessor produced bad rows.

Notebook 04 — Look at the per-flight probability plot in §7. The blue $P(\text{fault})$ line should be near zero during healthy portions and cross the red operational threshold roughly at (or slightly before) the pink ground-truth fault region. If $P(\text{fault})$ is flat 0.5 everywhere, the model didn't train; if it's 1 everywhere, the class weight is wrong.

5.2 The numbers to look at

- Notebook 01: `spec_energy_2_5kHz` should grow by roughly 50× or more from first to last snapshot. Reported in the printed table.
 - Notebook 02: `cross_max_asym` mean healthy vs mean fault, and the ratio. Should be at least 5×. On synthetic it's 40×.
 - Notebook 03: EoL cycle count per battery, printed in §2. Should be in the 130-160 range on synthetic (matches NASA's real dataset which sees ~168 for B0005).
 - Notebook 04: mean CV AUC (target > 0.85 on real data, > 0.95 on synthetic), and the top-5 feature importance list. Top-3 should be `cross_*` features.
-

Part 6 — What’s next after Phase 3a

6.1 Notebook 05 — Model B Gaussian process

The next notebook trains a Gaussian process on the battery features from Notebook 03. GP is the right model shape for two reasons: it outputs a full posterior distribution (not a point estimate), and it handles small-data regimes gracefully (we have only 4 cells worth of aging curves in NASA — an XGBoost model would overfit).

The specific GP kernel we’ll use is a Matérn 5/2 over (`cycles_elapsed`, `capacity_slope`, `r_slope`, `soh`, `internal_r_ohm`, `peak_temp_c`). That gives us smooth aging trajectories (Matérn 5/2 is twice-differentiable — matches the underlying physics), with the model automatically expanding the uncertainty band in regions of feature space where training data is sparse.

Success criterion: on held-out cells, the GP’s 90% credible interval should contain the ground-truth RUL curve at least 90% of the time. If it does, the twin can honestly report “flights remaining: 47 ± 12 ” and know that number is calibrated.

6.2 Notebook 06 — Worker integration

This is where the two model files (`bearing_wear.pkl` and `battery_rul.pkl`) get wired into `twin_worker.py`. The worker grows a new `ml_runner.py` (sibling to `fmu_runner.py`), which:

- Loads both `.pkl` and `.json` files at start.
- Validates the feature schema.
- Maintains a rolling telemetry buffer for the ALFA feature extractor.
- Calls both models at each tick.
- Publishes ML-derived predictions on the existing MQTT topics.
- Falls back cleanly to Phase 1c analytical predictors if any model fails to load.

The notebook doubles as an integration test: it replays a labelled ALFA fault flight through the SITL bridge → worker → ML layer, then verifies the alert fires on the dashboard within N seconds of the labelled fault onset.

6.3 Phase 3c and beyond

Once notebooks 05 and 06 are in, Phase 3 is essentially closed. The remaining work is (in order):

- **Domain adaptation to real fleet data.** Retrain both models on ArduPilot / PX4 logs from a design-partner operator once the Phase 2 conversations produce one.
- **Continuous re-training.** Every month, retrain on the last 90 days of fleet data and A/B against the incumbent model. Only ship if AUC improves and false-positive rate doesn’t grow.
- **Phase 4 (Real UAV).** By then the twin already has production-grade physics and production-grade ML. Real ESC telemetry replaces SITL’s simulated ESC telemetry; nothing else changes.

Part 7 — Reference

7.1 File map

uav-twin/notebooks/	
├─ README.md	you have this
├─ requirements.txt	you have this
├─ .gitignore	
├─ 01_ims_bearing_exploration.ipynb	Notebook 01 – established shape of bearing wear
├─ 02_alfa_fault_flights.ipynb	Notebook 02 – established shape of UAV fault
├─ 03_nasa_battery_aging.ipynb	Notebook 03 – established shape of Li-ion aging
├─ 04_model_a_xgboost.ipynb	Notebook 04 – first trained model
├─ 05_model_b_gp.ipynb	Notebook 05 – GP for battery
├─ 06_worker_integration.ipynb	Notebook 06 – wire ML into the twin worker
├─ utils/	
│ └─ nasa_ims_loader.py	synth + real data + feature extraction
│ └─ alfa_loader.py	same
│ └─ nasa_battery_loader.py	same
│ └─ (more helpers as needed)	
└─ data/	
│ └─ ims_features/*.parquet	Notebook 01 output
│ └─ alfa_features/*.parquet	Notebook 02 output → Notebook 04 input
│ └─ battery_features/*.parquet	Notebook 03 output → Notebook 05 input

And the model files the worker consumes:

uav-twin/worker/models/	
├─ UAVPropulsion.mo	Modelica source (Phase 1c)
├─ UAVPropulsion.fmu	compiled FMU (Phase 1c)
├─ bearing_wear.pkl	Model A (Phase 3b, produced by Notebook 04)
├─ bearing_wear.json	Model A config (operational threshold=0.5)
├─ battery_rul.pkl	Model B (Phase 3b, produced by Notebook 05)
├─ battery_rul.json	Model B config

7.2 Datasets used

Dataset	Size	Origin	Link
NASA IMS Bearing	~1 GB	Univ. Cincinnati IMS Center, mirrored on NASA PCoE	https://www.kaggle.com/datasets/vinayak123tyagi/bearing-dataset
ALFA UAV faults	~2 GB	CMU AirLab	https://github.com/castacks/alfa-dataset
NASA Li-ion battery aging	~500 MB	NASA PCoE	https://data.nasa.gov/dataset/li-ion-battery-aging-datasets

All permissively licensed; none blocks a future Apache-2.0 publication.

7.3 Glossary of terms

- **Feature** — a single number extracted from raw telemetry (e.g., `cross_max_asym = 5.5 A`). The model consumes a vector of features, not raw telemetry.
 - **Window** — a fixed-length time slice of telemetry (2 s at 10 Hz = 20 samples per motor). One window → one feature vector → one prediction.
 - **AUC** — area under the ROC curve. Ranges 0.5 (random) to 1.0 (perfect). Rule of thumb: > 0.85 is production-worthy, > 0.95 is very strong.
 - **Precision** — of the windows the model flagged as faults, what fraction really were faults. If precision is 0.9, one in ten alerts is false.
 - **Recall** — of the actual fault windows, what fraction the model caught. If recall is 0.99, one in a hundred faults is missed.
 - **Operational threshold** — the $P(\text{fault})$ cutoff at which the twin fires an alert. Chosen to hit a precision target, not to maximise AUC.
 - **Cross-validation (K-fold)** — split the data into K parts, train on K-1, test on 1, rotate K times. Averages out one-fold luck.
 - **Group K-fold** — same idea, but groups (in our case flights) never straddle the train/test split. The honest way to evaluate temporal ML.
 - **XGBoost** — gradient-boosted decision trees. Fast, interpretable via feature importance, well-behaved with tabular data. The right first choice for tabular ML.
 - **Gaussian process (GP)** — a nonparametric Bayesian model that outputs a full posterior over predictions. Good for small-data regimes where you need calibrated uncertainty.
 - **RUL — Remaining Useful Life** — the twin’s headline output. Measured in flights, cycles, or hours depending on the component.
 - **SoH — State of Health** — capacity remaining vs when new (%). Batteries. Reaches EoL at ~70-80% depending on chemistry.
 - **SoC — State of Charge** — battery charge level (%). Not the same as SoH.
 - **MCSA — Motor-Current Signature Analysis** — using motor current spectrum to detect mechanical faults. The bridge from vibration data (Notebook 01) to what a UAV ESC actually reports.
-

Appendix A — Completion status and achieved metrics

As of 15 September 2026, every notebook in this study guide is complete, both trained-model pickles are in the worker container, and the whole Phase 3c stack is verified end-to-end. This appendix records what was actually achieved against each notebook’s success criterion, and where the loose threads landed.

A.1 Per-notebook status

Notebook	Success criterion	Achieved (synthetic)	Real-data target	Status
01 IMS bearing	spec_energy_2_5kHz70× grows $\geq 50\times$ from healthy to failed		(deferred to real accelerometer data — not on our UAV)	Done, reference only
02 ALFA fault flights	cross_max_asym healthy/fault ratio $\geq 5\times$	$\sim 40\times$	$\geq 5\times$ on real ArduPilot logs	Done
03 NASA Li-ion	EoL cycle count in 130-160 range	Matches	—	Done
04 Model A XGBoost	Mean CV AUC ≥ 0.95 (synthetic) or ≥ 0.85 (real)	0.9998 , std 0.00019	≥ 0.85 on real fleet data	Done
05 Model B GP	90% CI covers ground-truth $\geq 90\%$ on held-out cells	92.4% coverage; RMSE 46.5 flights; MACE 0.032	$\geq 90\%$ on real cells	Done
06 Worker integration	Both models load in the running worker; MQTT topics stream	yes verified on docker compose --profile sitl up	(same)	Done

A.2 The three Phase 3c calibration fixes

Three issues surfaced during the Phase 3c integration session and were closed the same evening:

1. **Model A alert threshold too low.** Notebook 04 wrote `operational_threshold = 0.000673` — the F1-optimal value on training data where classes were nearly separable. On real telemetry the twin fired `alert=true` on every 2-second window because XGBoost’s minimum output floor sat above that threshold. Fixed by pinning the operational threshold to 0.5 (the natural probability decision boundary) in `bearing_wear.json`, with the notebook value preserved as `operational_threshold_notebook` for reproducibility.
2. **Model B feature schema mismatch.** `_FlightHistory.features()` in `twin_worker.py` originally returned an 8-key dict (my initial guess at the GP’s inputs). `battery_rul.json` actually names 6 keys: `cycles_elapsed`, `soh`, `internal_r_ohm`, `peak_temp_c`, `capacity_slope`, `r_slope`. `ml_runner.battery_rul()` silently caught the `KeyError` on `cycles_elapsed` and returned `unavailable: true`. Fixed by rewriting `_FlightHistory.features()` to emit exactly those six keys.
3. **SITL RPM stays at zero.** ArduPilot SITL’s motor model is thrust-only, so `ESC_TELEMETRY_1_T0_4.rpm` never rises above 0 even in cruise. The XGBoost feature

cross_min_rpm therefore carried no signal. Fixed by synthesizing RPM in bridge.py from $\text{throttle} \times V_{\text{pack}} \times \text{MOTOR_KV} \times \text{MOTOR_ETA}$ whenever the reported RPM is zero and the vehicle is armed. Real ESCs on a real UAV will report actual RPM directly and the synthesis path stops firing.

A.3 Two encoding traps worth naming

1. **Windows-1252 (CP1252) source files fail Python's UTF-8 parse.** Some editors saved twin_worker.py and ml_runner.py with em-dashes encoded as byte 0x97 (a valid CP1252 em-dash, an invalid UTF-8 lead byte). Python 3 refused to load them. Both files are now pinned pure-ASCII (`sum(1 for c in b if c > 127) == 0`) as a belt-and-braces guard.
2. **PowerShell 5.1 doesn't accept Windows-1252 as an -Encoding value.** The recovery snippet had to route through .NET directly: `[System.Text.Encoding]::GetEncoding(1252).GetContent -Encoding Windows-1252` call fails silently, leaves \$content unset, and if the follow-up WriteAllText passes a null \$content, PowerShell coerces to empty string and the file becomes 0 bytes.

A.4 Verification log lines (15 September 2026)

Worker startup:

```
[worker] starting UAV_ID=quad-01 MQTT=mosquitto:1883 influx=ok
[worker] telemetry topic: uav/quad-01/telemetry/state
[fmu_runner] loaded FMU from /app/models/UAVPropulsion.fmu
[ml_runner] loaded bearing_wear.pkl (xgboost, 28 features)
[ml_runner] loaded battery_rul.pkl (gp, 6 features)
[ml_runner] modes: {'model_a': 'xgboost', 'model_b': 'gp'}
[worker] connected rc=No error. (physics=fmu, ml={'model_a': 'xgboost', 'model_b': 'gp'})
[worker] subscribed to uav/quad-01/telemetry/state
```

MQTT snapshot of one tick:

```
uav/quad-01/telemetry/state {"phase":"cruise","throttle":0.19,
                             "battery":{"pack_v":16.8,"pack_i_a":21.25,"soc":0.58,...},
                             "motors":[{"rpm":2441.9,"current_a":0.8,"temp_c":32.0,...},...],
                             "_source":"mavlink"}
uav/quad-01/twin/state      {"physics":"fmu","SoC":1.0,"SoH":1.0,"V_pack":16.8,
                             "motor_temps_c":[32.67,32.67,32.67,32.67],
                             "motor_damage":[1.21e-07,1.21e-07,1.21e-07,1.21e-07]}
uav/quad-01/twin/anomaly   {"p_fault":0.00138,"alert":false,"mode":"xgboost"}
uav/quad-01/twin/rul       {"mode":"gp","flights_observed":0,"unavailable":true,
                             "reason":"warming_up"}
```

The warming_up state clears once ~5 Ah of pack throughput accumulates (~14 min of continuous cruise), at which point twin/rul starts emitting real median, p05, p95 in units of flights remaining.

A.5 Where to go next

The operational reference for the built twin is `phase-3c-complete.md` — every service, every topic schema, every env var, the verification runbook. This study guide (Parts 1-7) remains the pedagogical walkthrough.

The next milestones, ordered by cost-to-value: dashboard update, GitHub publication, Phase 4 real-UAV retrain, Model C anomaly detector, continuous re-training pipeline. See §4 of `technical-overview.md` for detail.

Read the notebooks alongside this guide when you have an hour. The plots are the story; the code is the receipt.